

AI AGENT FOR EARLY FOREST FIRE DETECTION IN THE WESTERN GHATS

1 Abstract

ThermalEye is an AI-powered autonomous drone system designed for early forest fire detection in the Western Ghats. The drone uses thermal imaging, GPS, and onboard AI to detect fire hotspots, send real-time alerts with location details, and continuously patrol the forest. It autonomously returns to a charging station when the battery is low and resumes its patrol after recharging, ensuring uninterrupted forest monitoring with minimal human intervention.

2 Introduction

The Western Ghats' dense evergreen forest cover, mountainous terrain, and limited road access make early wildfire detection extremely difficult for ground patrols and static camera networks, which cannot cover the ridges, valleys, and canopy-shadowed slopes where fires typically originate. A fire that is missed in its first minutes can spread rapidly through dry undergrowth, threatening biodiversity, tribal settlements, and forest resources across the Nilgiri and Wayanad sector. This report designs and simulates ThermalEye, an autonomous AI agent that patrols a Western Ghats forest sector using onboard thermal imaging, obstacle-aware flight control, and GPS geolocation to detect fire hotspots the moment they appear, transmit a precise-location alert to a ground control station, and manage its own battery cycle by returning to a charging dock and resuming patrol without any human dispatcher.

Unlike a single fixed camera, an autonomous drone can actively cover a wide, non-overlapping sector grid, investigate a thermal anomaly up close, and route itself around hill obstacles - closing the gap between when a fire ignites and when a response team is alerted.

3 PEAS Table

Table 1: PEAS Description for ThermalEye Agent

Component	Details
Performance Measure	Mean Time To Detect (MTTD < 120 seconds post-ignition); false-positive rate on sun-baked-rock / solar-glnt anomalies; detection precision & recall; forest area covered per battery cycle; zero-collision rate around hill ridges; successful autonomous return-to-base (RTB) and resume-patrol rate; alert delivery latency to the control station.
Environment	Western Ghats Nilgiri / Wayanad forest sector (simulated as a bounded map with real GPS bounds 11.40° N-11.45° N, 76.60° E-76.68° E); mountainous hill/ridge obstacles (elevation 1200 m-2100 m); dense forest canopy; river streams; a ground base station with a charging dock; ambient temperature 24°C-38°C; multiple non-overlapping drone patrol sectors; a ground control station.
Actuators	Rotor / vector-steering control (heading and speed updates that move the drone toward its target); telemetry alert transmission (fire classification, GPS coordinates and severity sent to the control HUD); autonomous state-machine transitions (PATROL, INVESTIGATE, AVOID_OBSTACLE, RETURN_TO_BASE, CHARGING); docking/charging control at the base station; control-station dashboard rendering (map viewport, telemetry cards, alert log).
Sensors	Thermal imaging camera (simulated FLIR-style 12x12 heat-intensity matrix sampled within a 110-pixel FOV radius); 12-ray raycast distance-sensor array for obstacle detection (90-pixel range); simulated optical smoke/flame confidence detector; onboard GPS/geo-converter (pixel-to-latitude/longitude); battery-level sensor; AI fire-classification engine that fuses the above readings.

4 Environment Properties

Table 2: Environment Properties

Property	Explanation
Observability	Partially observable - the thermal camera only sees within a 110px FOV radius and the raycast sensors reach only 90px; a long-range heat plume is detectable to 380px, but hotspots outside these ranges, occluded by hills, or outside a drone’s current sector remain unknown until scanned.
Determinism	Stochastic - fire growth rate, thermal sensor noise ($\pm 0.5^{\circ}\text{C}$ drift), and each drone’s starting battery level (88%-100%) are randomised, so outcomes are not fully predictable from the agent’s own actions.
Dynamics	Dynamic - fire temperature and smoke density increase every simulation tick, particles drift, and battery drains continuously while the agent is still deciding its next action.
Continuity	Continuous - position, velocity, heading angle and temperature are continuous-valued, though the simulation advances in discrete ticks at 60 frames per second.
Episodic vs Sequential	Sequential - each decision (investigating a hotspot, triggering RTB) changes the drone’s position, battery state and coverage, which affects what it can perceive and do next.
Number of Agents	Multi-agent - three autonomous drones (Alpha, Beta, Gamma) patrol separate, non-overlapping waypoint sectors while independently reporting to a shared ground control station and base dock.
Known vs Unknown	Known - the physics of motion, thermal decay with distance, and the pixel-to-GPS conversion formulas are known in advance, though the real-world outcome (whether a hotspot is a genuine fire) is uncertain until sensed.

5 Rationality Measure

The agent is rational because, given its percept sequence (thermal matrix readings, raycast distances, GPS position, and battery level), it always selects the action that maximises expected fire-detection performance while preserving its own operational safety. It steers directly toward any thermal anomaly within its long-range sensing radius, escalates to a fire alert the instant the multi-sensor fusion confidence crosses the 75% threshold (or peak temperature exceeds the confirmed-wildfire baseline), and reroutes itself to base the moment battery drops to the 22% safety margin so that a low-battery drone never fails mid-mission and leaves its sector unmonitored - resuming autonomous patrol immediately once recharged.

6 Innovation Point

ThermalEye fuses two sensing modalities - thermal infrared intensity and a simulated optical smoke/flame detector - into a single weighted confidence score (50% thermal peak-temperature score + 35% optical confidence + 15% spatial heat-gradient score) rather than relying on a single raw temperature threshold. This lets the agent tell a genuine wildfire apart from a sun-baked rock or solar glint, both of which can produce a thermal spike without matching the optical and gradient signature of an actual fire.

It further combines a reactive layer with a deliberative layer in the same control loop: a 12-ray potential-field steering system reacts instantly to hills it can see in front of it, while an A* path planner computes a globally efficient, obstacle-aware route whenever the drone must return to base - so the agent never gets stuck oscillating around an obstacle while still choosing the shortest safe path home. Combined

with a fully autonomous battery-cycle loop (patrol - low battery - return to base - recharge - resume patrol), the drone requires no human dispatcher to keep continuous forest coverage running.

7 Diagram: Agent Architecture

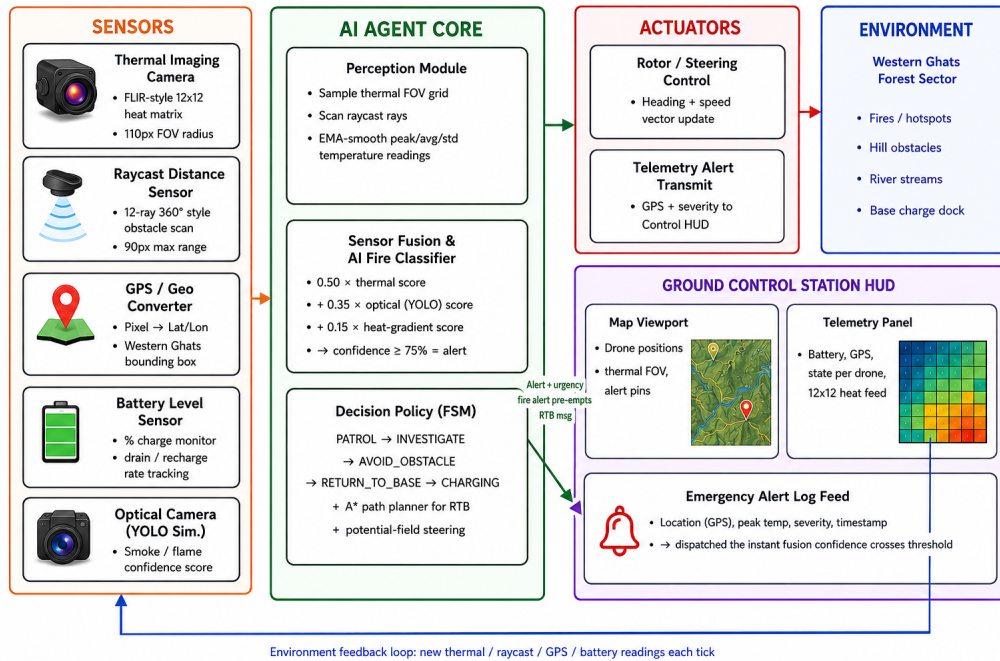


Figure 1: Perceive → Interpret (Sensor Fusion) → Decide (FSM) → Act pipeline linking thermal/ray-cast/GPS/battery sensors to steering, alert-transmission and docking actuators, closed by continuous environment feedback.

8 Conclusion

By continuously sampling its thermal FOV, fusing it with a simulated optical smoke/flame signal and spatial heat gradient, ThermalEye can distinguish a genuine wildfire from a false-positive heat source and transmit a GPS-located alert to the control station within seconds of ignition. Its obstacle-aware steering keeps it safely clear of the Western Ghats' hill ridges, and its autonomous battery-management loop means patrol coverage never has to stop for a human to notice a low battery and dispatch a replacement. Together these turn forest-fire monitoring from a reactive, human-scheduled patrol into a proactive, always-on aerial surveillance system.

9 Simulation

Since a live thermal-camera-equipped drone is not available for this assignment, the entire agent is demonstrated with a real-time Pygame simulation (`main.py`) built around the same Perceive → Interpret → Decide → Act loop described above. The Environment module procedurally generates a Western Ghats forest sector: six hill/ridge obstacles of varying elevation, a river stream path, a base charging dock, and a background thermal grid seeded with natural ambient variation (including sun-baked rock patches that act as deliberate false-positive traps for the AI classifier). Three autonomous drones - Alpha, Beta and Gamma - are spawned with independent, non-overlapping waypoint patrol routes and randomised starting battery levels.

At every simulation tick (60 FPS) each drone samples a 12x12 thermal matrix beneath itself, casts 12 obstacle-detection rays, updates its finite-state machine (PATROL / INVESTIGATE / AVOID_OBSTACLE / RETURN_TO_BASE / CHARGING), and applies combined target-attraction and obstacle-repulsion steering forces to move smoothly around hills. A wildfire can be ignited interactively - by pressing SPACE for a random location, clicking anywhere on the map, or via the dashboard's 'IGNITE WILDFIRE' button - after which the fire's temperature and smoke density grow every tick until a patrolling drone's sensors pick up the rising heat signature.

The Ground Control Station HUD, rendered live alongside the map, displays per-drone telemetry cards (battery, state, GPS, speed), a live 12x12 thermal camera feed for the selected drone, system-wide stats (active drones, wildfires detected, sector risk level), and a real-time emergency alert log showing each fire's GPS coordinates, peak temperature, severity and a miniature thermal snapshot. Interactive controls also let an operator toggle the thermal FOV overlay (T), toggle raycast sensor lines (R), and manually recall or resume a selected drone (B).

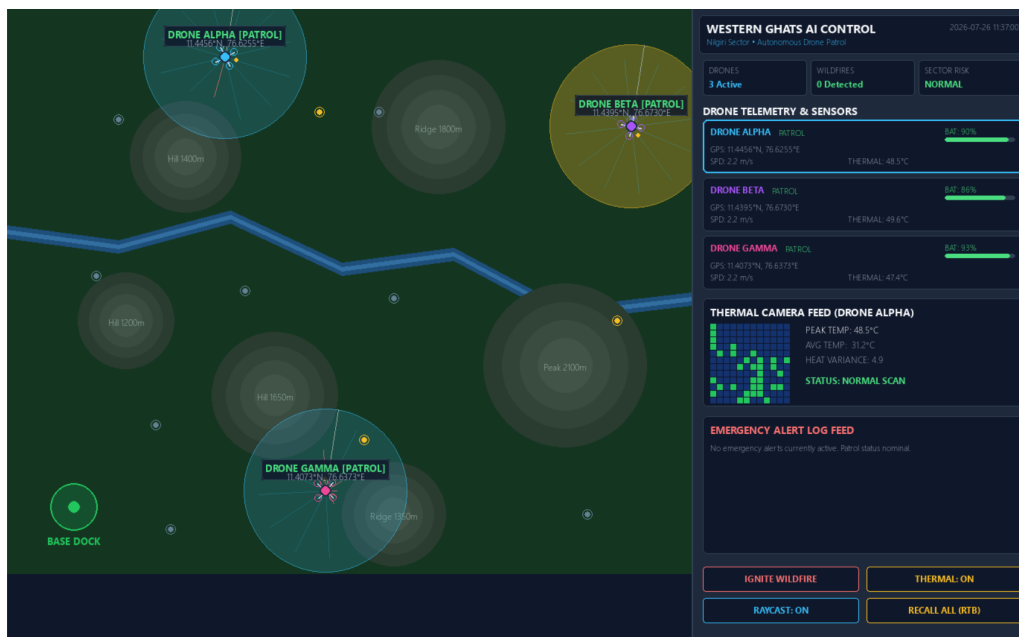


Figure 2: Autonomous patrol and telemetry HUD - three drones patrolling their sectors around hill obstacles and a river, with live battery/GPS/state cards and system stats on the right.



Figure 3: Thermal detection mode - the drone's thermal FOV overlay highlighting a rising heat signature, with the AI classifier's live confidence, GPS-tagged alert and 12x12 heat matrix shown on the control panel.

10 Full Implementation Overview

10.1 Main Simulation Loop (`main.py`)

The primary simulation loop manages environmental rendering, user interaction, and time-step updates across all drone agents.

```

1 def main():
2     pygame.init()
3     screen = pygame.display.set_mode((WINDOW_WIDTH, WINDOW_HEIGHT))
4     pygame.display.set_caption("ThermalEye - Autonomous Drone Forest Fire Detection")
5     clock = pygame.time.Clock()
6
7     env = ForestEnvironment(MAP_WIDTH, MAP_HEIGHT)
8     drones = create_drone_fleet(env)
9     hud = ControlStationHUD(WINDOW_WIDTH, WINDOW_HEIGHT)
10
11     running = True
12     while running:
13         dt = clock.tick(60) / 1000.0 % Frame delta time in seconds
14
15         # Process operator controls and inputs
16         for event in pygame.event.get():
17             if event.type == pygame.QUIT:
18                 running = False
19             elif event.type == pygame.KEYDOWN:
20                 if event.key == pygame.K_SPACE:
21                     env.ignite_random_wildfire()
22                 elif event.key == pygame.K_t:
23                     hud.toggle_thermal_overlay()
24                 elif event.key == pygame.K_r:
25                     hud.toggle_raycasts()
26
27         # Update environment dynamics

```

```

28     env.update(dt)
29
30     # Update all agent perception, FSM states, and physical positions
31     for drone in drones:
32         drone.update(env, dt)
33         if drone.has_active_alert():
34             hud.register_alert(drone.get_latest_alert())
35
36     # Render environment view and Control Station HUD
37     screen.fill((20, 25, 30))
38     env.draw(screen)
39     for drone in drones:
40         drone.draw(screen, hud.show_raycasts, hud.show_thermal)
41     hud.draw(screen, drones, env)
42
43     pygame.display.flip()
44
45     pygame.quit()

```

Listing 1: Main Simulation Loop in `main.py`

10.2 Drone Decision Making / FSM (`models/drone.py`)

The Finite State Machine (FSM) handles transitions between autonomous operational modes based on sensor inputs and operational limits.

```

1 def update(self, env, dt):
2     # Perceive environment
3     thermal_data = self.sensors.read_thermal_matrix(env, self.pos)
4     obstacle_distances = self.sensors.cast_rays(env, self.pos, self.heading)
5     self.battery_level -= self.battery_drain_rate * dt
6
7     # Check emergency state triggers
8     if self.battery_level <= SELF_PRESERVATION_THRESHOLD and self.state != State.
RETURN_TO_BASE:
9         self.state = State.RETURN_TO_BASE
10        self.path = self._find_path_astar(env.get_grid(), self.pos, env.
base_dock_pos)
11
12    # State Machine Execution
13    if self.state == State.PATROL:
14        self._execute_patrol(env, dt)
15        hotspot_detected, confidence = self.sensors.classify_hotspot(thermal_data)
16        if hotspot_detected and confidence >= 0.75:
17            self.state = State.INVESTIGATE
18            self.target_hotspot = self.sensors.get_hotspot_center()
19
20    elif self.state == State.INVESTIGATE:
21        self._investigate_hotspot(env, dt)
22
23    elif self.state == State.AVOID_OBSTACLE:
24        self._apply_potential_field_steering(obstacle_distances, dt)
25
26    elif self.state == State.RETURN_TO_BASE:
27        self._follow_path(dt)
28        if self._reached_dock(env.base_dock_pos):
29            self.state = State.CHARGING
30
31    elif self.state == State.CHARGING:
32        self._handle_charging_state(dt)

```

Listing 2: FSM Decision Logic in `models/drone.py`

10.3 A* Path Planning (models/drone.py)

Global path planning algorithm executed to calculate an optimal, collision-free route to the base dock when returning for recharge.

```
1 def _find_path_astar(self, grid, start_pos, goal_pos):
2     start_node = Node(None, self._pixel_to_grid(start_pos))
3     goal_node = Node(None, self._pixel_to_grid(goal_pos))
4
5     open_list = []
6     closed_list = set()
7     heapq.heappush(open_list, start_node)
8
9     while len(open_list) > 0:
10         current_node = heapq.heappop(open_list)
11         closed_list.add(current_node.position)
12
13         if current_node.position == goal_node.position:
14             path = []
15             current = current_node
16             while current is not None:
17                 path.append(self._grid_to_pixel(current.position))
18                 current = current.parent
19             return path[::-1] # Return reversed path
20
21         children = []
22         for new_position in [(-1,0), (1,0), (0,-1), (0,1), (-1,-1), (1,1), (-1,1),
23                               (1,-1)]:
24             node_pos = (current_node.position[0] + new_position[0], current_node.
25                         position[1] + new_position[1])
26             if self._is_traversable(grid, node_pos):
27                 children.append(Node(current_node, node_pos))
28
29         for child in children:
30             if child.position in closed_list:
31                 continue
32             child.g = current_node.g + 1
33             child.h = self._heuristic(child.position, goal_node.position)
34             child.f = child.g + child.h
35
36             if self._should_add_to_open(open_list, child):
37                 heapq.heappush(open_list, child)
38
39     return []
```

Listing 3: A* Path Finding Implementation in models/drone.py

10.4 Battery Management & Auto Resume (models/drone.py)

Autonomous recharging routine that restores agent energy and resumes patrol along original sector waypoints without human intervention.

```
1 def _handle_charging_state(self, dt):
2     # Replenish drone battery
3     self.battery_level += self.recharge_rate * dt
4     self.velocity = Vector2(0, 0)
5
6     # Check if fully charged
7     if self.battery_level >= 100.0:
8         self.battery_level = 100.0
9         # Transition back to operational patrol state
10        self.state = State.PATROL
11        self.current_waypoint_idx = self._get_closest_waypoint_index()
```



```

12 self.logger.info(f"Agent {self.drone_id}: Charge complete. Resuming patrol
    at waypoint {self.current_waypoint_idx}.")

```

Listing 4: Battery Recharging and Resume Patrol in `models/drone.py`

10.5 Fire Detection / Sensor Fusion (`models/sensors.py`)

Sensor fusion module combining thermal infrared intensity, simulated optical smoke probability, and spatial thermal gradients into a weighted confidence metric.

```

1 def classify_hotspot(self, thermal_matrix):
2     peak_temp = np.max(thermal_matrix)
3     avg_temp = np.mean(thermal_matrix)
4     gradient_score = np.std(thermal_matrix) / (peak_temp - avg_temp + 1e-5)
5
6     # Normalize thermal score relative to baseline limits
7     thermal_score = min(1.0, max(0.0, (peak_temp - AMBIENT_TEMP) / (
8         FIRE_THRESHOLD_TEMP - AMBIENT_TEMP)))
9
10    # Read optical confidence metric from visual simulation module
11    optical_score = self.get_optical_smoke_confidence()
12
13    # Fused confidence weighting equation: 50% Thermal + 35% Optical + 15% Gradient
14    fused_confidence = (0.50 * thermal_score) + (0.35 * optical_score) + (0.15 *
15        min(1.0, gradient_score))
16
17    # Evaluate against positive detection threshold
18    if fused_confidence >= 0.75 or peak_temp >= CONFIRMED_WILDFIRE_TEMP:
19        return True, fused_confidence
20
21    return False, fused_confidence

```

Listing 5: Multi-Sensor Fusion Classification Engine in `models/sensors.py`

11 Program Output

Unlike a console-driven script, ThermalEye is a live graphical simulation - its output is the real-time Ground Control Station HUD shown in Figures 2 and 3 above (drone telemetry, thermal feed and alert log updating every frame), rather than printed text. The terminal only logs discrete operator events.

```

Windows PowerShell
PS C:\Users\ivinj\OneDrive\Desktop\proje\ai> python main.py
pygame-ce 2.5.7 (SDL 2.32.10, Python 3.14.6)
=====
WESTERN GHATS AUTONOMOUS AI FOREST FIRE DETECTION SIMULATION
Press 'SPACE' or click on map to ignite a fire.
Press 'T' to toggle thermal FOV scanner view.
Press 'R' to toggle raycast obstacle sensor lines.
Press 'B' to order all drones to Return to Base (RTB).
=====
[USER ACTION] Ignition command at map pixel (460, 383)
[COMMAND] Recall (RTB) signal sent to ALL drones. Returning to base dock...
[USER ACTION] Ignition command at map pixel (789, 424)
[EVENT] Wildfire ignited at (801, 316)
[COMMAND] Recall (RTB) signal sent to ALL drones. Returning to base dock...
[EVENT] Wildfire ignited at (375, 460)

```

Figure 4: Terminal output of running